

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Implementación de Robótica Inteligente	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil	
Módulo:	Actividad: Mini Reto Semana 1	
Fecha: 06 de abril del 2026		

Resumen

En este mini reto, primero fue necesario configurar el sistema operativo de la Jetson Nano, pues es la tarjeta principal del robot utilizado para esta práctica, este robot es proporcionado por Manchester Robotics, siendo este robot un Puzzle Bot. Una vez con el sistema operativo listo, se hicieron últimas configuraciones para Ubuntu, y teniendo esto listo, ahora si se configuraron las redes para la comunicación entre computador y robot, la cual es tipo Wifi, esto se hizo configurando un Hotspot. Teniendo ya la comunicación computador-robot, ahora sí se puede ejecutar el código de micro-ROS, el cual permite mover el robot de forma teleoperada.

Objetivos

El objetivo de la primera práctica es conocer el robot que se utilizará a lo largo del periodo, mismo que se usará en las siguientes prácticas. En este mini reto primeramente fue necesario comprobar que todos los elementos del robot Puzzle Bot funcionaran correctamente, al igual que la tarjeta SD, donde fue necesario preparar el sistema operativo de la Jetson Nano. Una vez teniendo listo esto, fue necesario configurar el Hotspot para la comunicación por Wifi entre la computadora y el robot. Finalmente, con el código proporcionado por Manchester Robotics, el robot debe ser teleoperado para cada uno de sus movimientos. Igualmente, es

importante mencionar que es necesario identificar la zona muerta en las velocidades del robot.

Introducción

En el ámbito de la robótica la integración de software y hardware resulta esencial y fundamental para el desarrollo de sistemas capaces de interactuar con el entorno ya sea de manera teleoperada o autónoma. Es por ello que este mini reto se enfocó en la configuración, conexión y control de un robot móvil llamado Puzzlebot el cual utiliza herramientas del ecosistema ROS 2, lo que permite que se pueda realizar una comunicación entre nodos y dispositivos.

Hablando específicamente del robot utilizado, el Puzzlebot es una plataforma robótica móvil de bajo costo elaborada por Manchester Robotics diseñada para facilitar el aprendizaje, desarrollo e implementación de aplicaciones robóticas. Su principal objetivo es democratizar la robótica permitiendo a los usuarios acceder a tecnologías avanzadas y desarrollar proyectos cada vez más complejos sin necesidad de cambiar de plataforma (Manchester Robotics, 2026). Además, el Puzzlebot es un sistema modular tipo plug & play que es capaz de integrar distintos componentes como sensores, unidades de procesamiento, actuadores, entre otros sistemas que lo convierten en una gran herramienta para el aprendizaje y la investigación.

Consecuentemente, el Puzzlebot Nvidia Jetson el cual se utilizó para este reto cuenta con diferentes componentes como una Nvidia Jetson Nano la cual permite ejecutar algoritmos avanzados de inteligencia artificial, visión por computadora y/o navegación autónoma. Asimismo, cuenta con una hackerboard basada en una ESP32 para algoritmos de control de bajo nivel y una cámara Raspberry Pi. De igual manera el Puzzlebot integra componentes como motores, encoders para la medición de velocidad y un sistema de control en la hackerboard que permite el movimiento del robot en función de comandos externos.

Tomando en cuenta lo anterior, para lograr la interacción con el robot se utiliza el protocolo SSH (Secure Shell), el cual permite acceder de forma remota al sistema del Puzzlebot a través de una red WiFi generada por el mismo dispositivo. A través de esta conexión es posible ejecutar comandos y lanzar procesos necesarios para la comunicación con ROS 2. Asimismo, la teleoperación del robot se realiza mediante Teleop Twist Keyboard la cual permite enviar comandos de velocidad lineal y angular a través de mensajes tipo Twist. Estos comandos son publicados en el tópico `/cmd_vel`, mientras que los sensores del robot generan información de retroalimentación en los tópicos `/VelEncL` y `/VelEncR`, permitiendo monitorear el comportamiento del robot en tiempo real.

Es por ello que en este mini reto nos permite comprender el funcionamiento del Puzzlebot, configurarlo para su buen funcionamiento, entender los fundamentos de la comunicación en sistemas robóticos y la interacción entre hardware y software en un robot móvil.

Solución del problema

Para dar solución a los objetivos del reto, la metodología comenzó con la preparación del sistema operativo de la Jetson Nano instalada en el Puzzlebot. El proceso se llevó a cabo desde una computadora externa con sistema operativo Windows 11, donde se utilizó el software BalenaEtcher para flashear la imagen comprimida proporcionada en una tarjeta microSD de 64 gigabytes. Una vez finalizado el proceso de escritura, la memoria se insertó en la placa para realizar el primer arranque y completar el asistente de configuración inicial de Ubuntu.

Configuración de credenciales y red

Una vez iniciado el entorno con las configuraciones por defecto, se personalizó el acceso creando un usuario llamado "Travieso" en lugar de utilizar las credenciales por defecto. A este perfil se le asignó una contraseña intencionalmente sencilla de recordar, con el objetivo de agilizar el flujo de trabajo al momento de ejecutar comandos con privilegios de administrador mediante la instrucción sudo. Posteriormente, se habilitó la comunicación inalámbrica configurando la Jetson como un punto de acceso Wi-Fi propio desde el gestor de redes del sistema. Esto permitió enlazar la computadora externa a la red local del robot y establecer una conexión remota estable a través de SSH utilizando el nuevo usuario creado.

Implementación y seguimiento del código

Tras acceder a la terminal de la Jetson vía SSH, la siguiente etapa consistió en inicializar la comunicación con el microcontrolador del Puzzlebot. Se ejecutó el agente de micro-ROS mediante la terminal apuntando al puerto serial del dispositivo usando el siguiente comando: `ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0 -b 115200` integrando así el hardware con el entorno de ROS2.

El seguimiento y validación del código se realizó solicitando la lista de tópicos activos en el sistema (`ros2 topic list`) para confirmar la correcta aparición de las rutas de control y retroalimentación, tales como `/cmd_vel`, `/VelEncL` y `/VelEncR`. Finalmente, se publicaron mensajes de tipo `geometry_msgs/msg/Twist` en el tópico de velocidad para verificar la respuesta de los motores. Esto se realizó mediante la ejecución del nodo

teleop_twist_keyboard (*ros2 run teleop_twist_keyboard teleop_twist_keyboard*). Con todos los nodos funcionando correctamente, el robot quedó listo para la fase de teleoperación.

Resultados

Primero que nada, se logró la creación de un hotspot para la conexión entre una computadora externa y la *Jetson* del Puzzlebot.

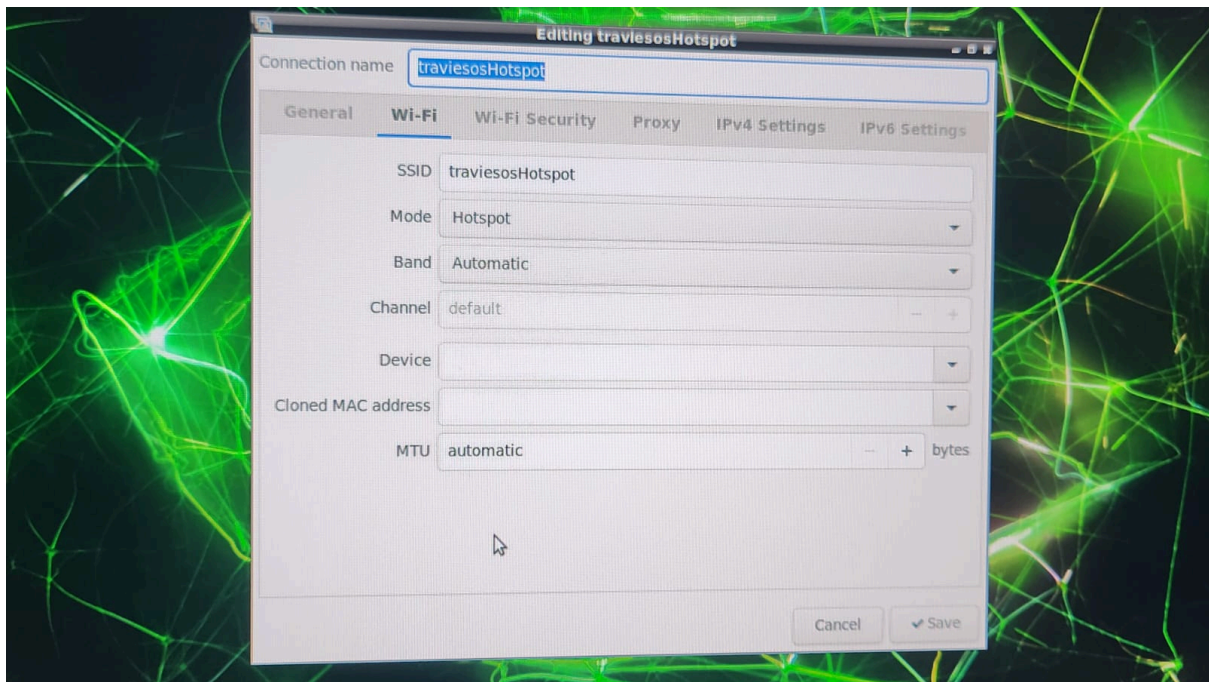


Imagen 1. Información general del hotspot creado.

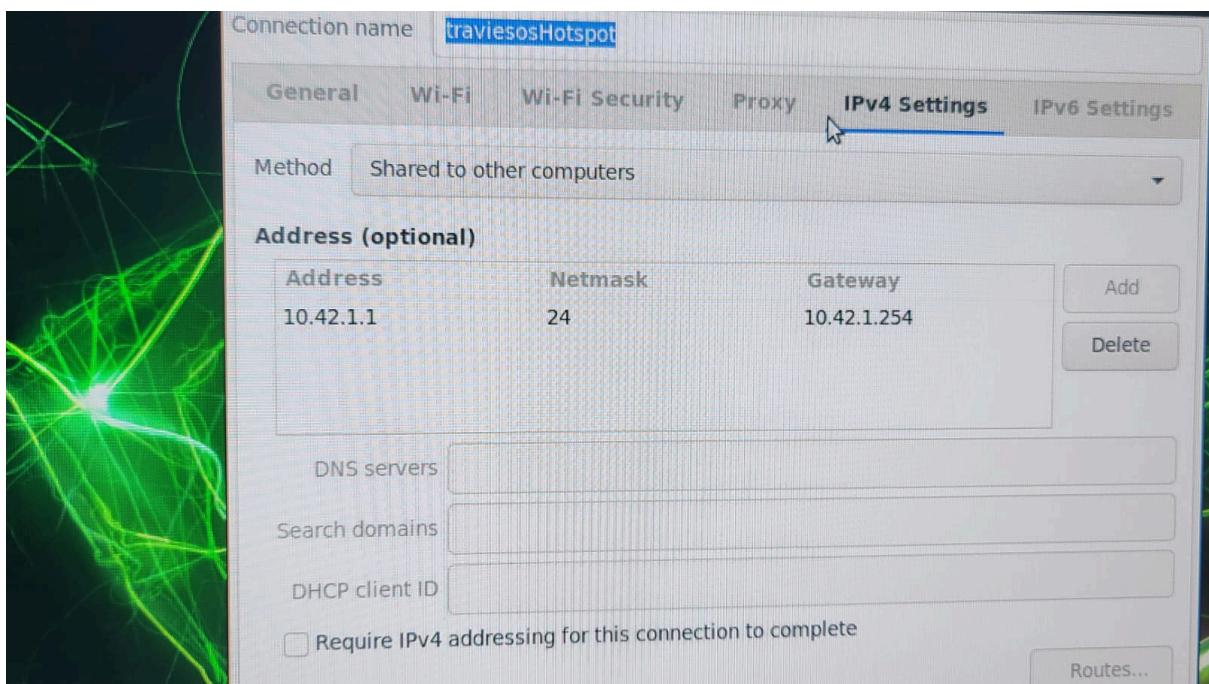


Imagen 2. Dirección IP del hotspot.

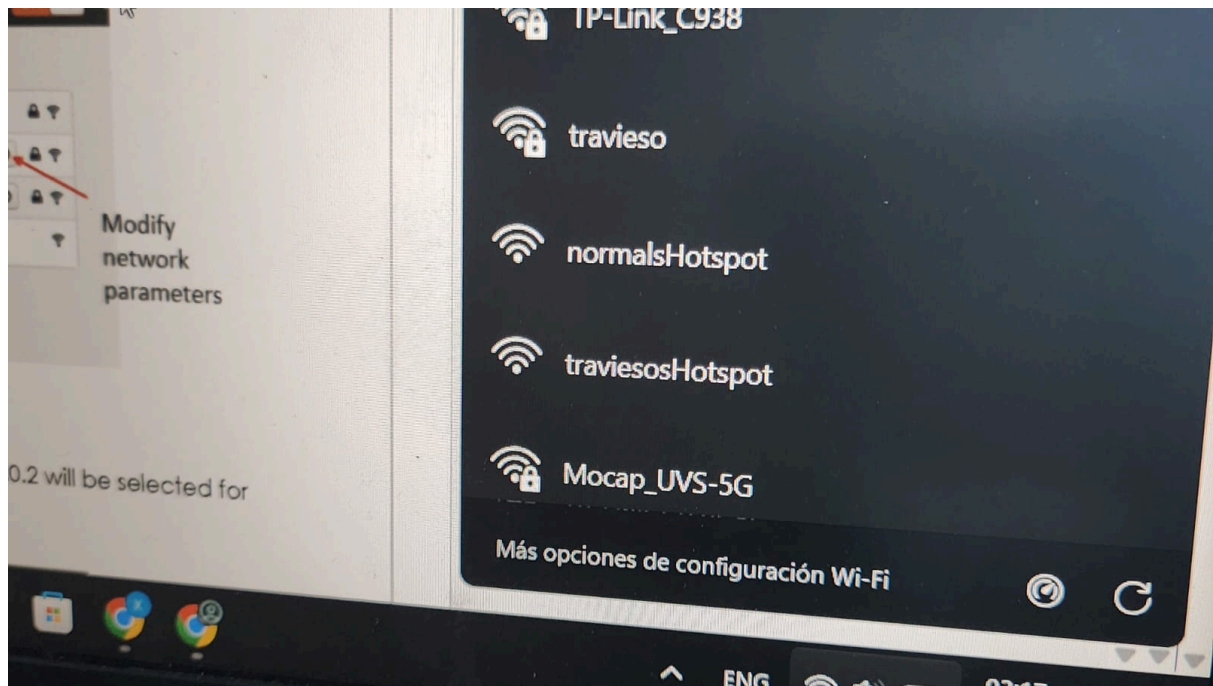


Imagen 3. Visualización del hotspot creado en la red.

En las **Imágenes 1, 2 y 3**, se puede observar la creación de un hotspot de nombre “traviesosHotspot”, con una dirección IP de 10.42.1.1. Esta red se utilizará para establecer conexión mediante WiFi a través de SSH con una computadora externa. En este caso, se utilizó una computadora con sistema operativo Ubuntu 22.04 para realizar la conexión. Primero se conectó la computadora al hotspot, y luego se conectó mediante SSH usando el usuario y contraseña previamente creados.

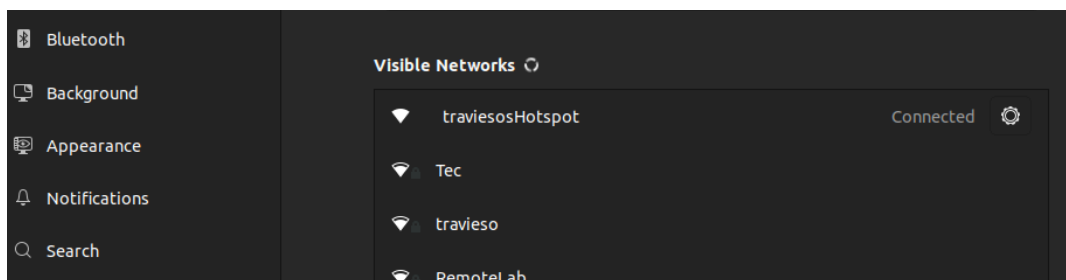


Imagen 4. Computadora conectada al hotspot.

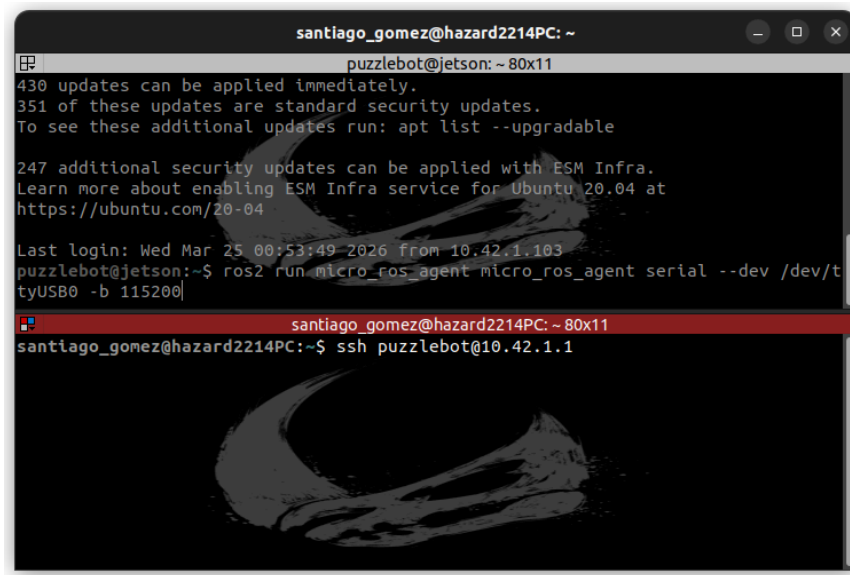
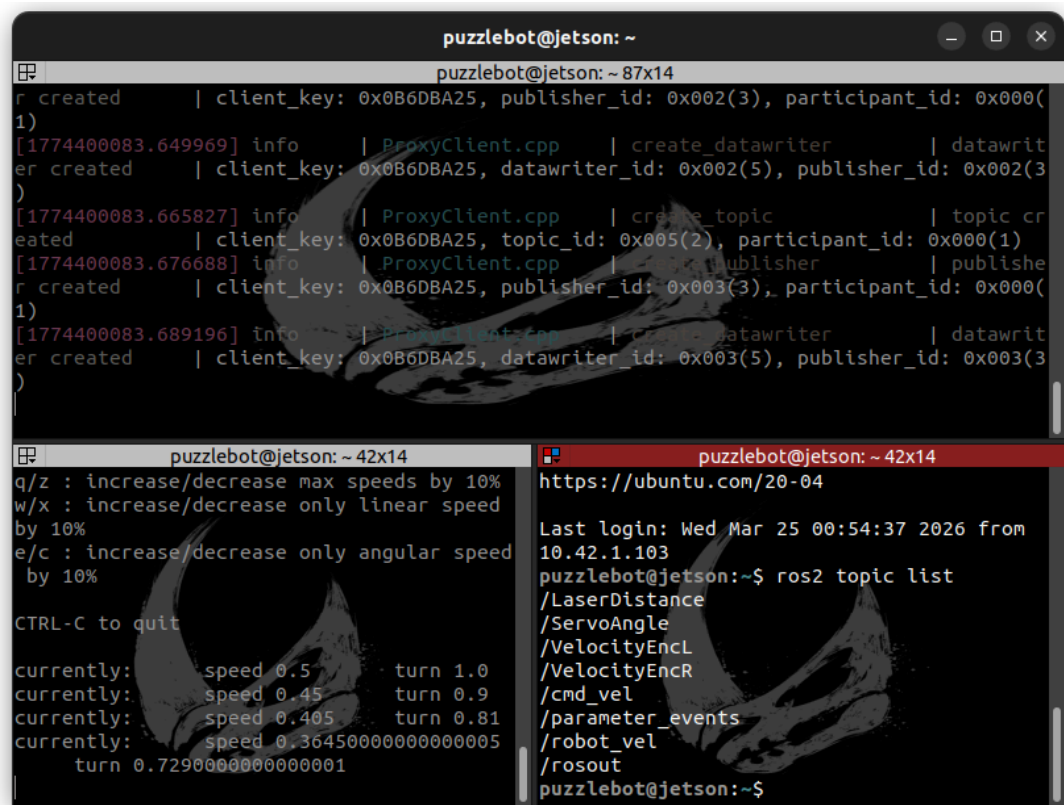


Imagen 5. Conexión por SSH mediante la terminal de la computadora.

En la **Imagen 4** se observa que la computadora está conectada por SSH a la red, con lo cual, puede detectar la señal IP de la *Jetson*. En la **Imagen 5**, se pueden ver dos terminales. La primera muestra que ya se ha conectado por SSH a la *Jetson*, y en la segunda se muestra el comando utilizado para hacer la conexión.

Para probar el funcionamiento del Puzzlebot, primero se inició el agente de micro-ROS para conectar con la Hackerboard montada en el robot. Como está conectado en el puerto serial `/dev/ttyUSB0`, se utilizó el comando `ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -b 115200`. Una vez inicializado, los motores se controlaron mediante el paquete de *teleop_twist_keyboard*. Mediante experimentación visual, se determinó que el máximo valor que se le puede mandar en la velocidad lineal en el tópico `/cmd_vel` es de 0.36. Esto se debe a que al exceder este valor, se producía un pico de corriente que provocaba que se apagara la *Jetson*.



The image displays three terminal windows from a remote SSH session on a Jetson device, with the prompt 'puzzlebot@jetson: ~'.

- Top Terminal (87x14):** Shows ROS2 initialization logs. It includes messages like 'r created', 'ProxyClient.cpp | create_datawriter', and 'datawriter created', along with client_key, publisher_id, participant_id, and datawriter_id values.
- Bottom Left Terminal (42x14):** Displays the 'teleop_twist_keyboard' node's control interface. It lists keyboard shortcuts: 'q/z : increase/decrease max speeds by 10%', 'w/x : increase/decrease only linear speed by 10%', and 'e/c : increase/decrease only angular speed by 10%'. It also shows 'CTRL-C to quit' and a table of current speeds and turns:

currently:	speed 0.5	turn 1.0
currently:	speed 0.45	turn 0.9
currently:	speed 0.405	turn 0.81
currently:	speed 0.36450000000000005	turn 0.72900000000000001
- Bottom Right Terminal (42x14):** Shows the output of the 'ros2 topic list' command, displaying a list of active topics:

```
/LaserDistance
/ServoAngle
/VelocityEnCL
/VelocityEncR
/cmd_vel
/parameter_events
/robot_vel
/rosout
```

Imagen 6. Visualización del funcionamiento del Puzzlebot.

En la **Imagen 6** se observan 3 terminales que están conectadas mediante SSH a la *Jetson*. La terminal superior muestra el agente de micro-ROS ya operativo. La terminal de la esquina inferior izquierda muestra el nodo de *teleop_twist_keyboard* ya iniciado para controlar al robot con el teclado de la computadora, en la cual también se muestra la velocidad actual lineal y angular actual con la que está operando el Puzzlebot. La terminal de la esquina inferior derecha muestran los tópicos que ya están inicializados actualmente, los cuales se iniciaron por el código de mico-ROS de la Hackerboard y del nodo de *teleop_twist_keyboard*.

En el siguiente video, se muestra el funcionamiento del Puzzlebot:

Conclusiones

En el desarrollo del presente mini reto se lograron cumplir los objetivos satisfactoriamente ya que fue posible configurar correctamente el Puzzlebot, establecer la comunicación entre la

computadora y la Jetson Nano mediante el protocolo SSH y finalmente realizar la teleoperación del robot utilizando ROS 2. Esto se evidencia en el correcto funcionamiento y configuración del sistema, permitiendo controlar el movimiento del robot a través de la computadora, comandos de velocidad y observando su respuesta en tiempo real.

Sin embargo, durante el proceso de configuración se presentó una dificultad con el módulo de red inalámbrica, el cual no generaba el hotspot necesario para la comunicación con la Jetson Nano y la computadora externa. Esta situación primeramente impidió establecer la conexión remota, lo que afectó temporalmente el flujo del trabajo. Para resolver este problema primeramente se intentó con diferentes módulos wifi, percatándose que el que se tenía no era compatible con el sistema operativo. Es por ello que fue necesario utilizar otro módulo wifi externo compatible, lo que permitió restablecer la comunicación y continuar con el desarrollo del mini reto.

A pesar de esta limitación temporal, se logró no solo completar la teleoperación del robot Puzzlebot, sino también identificar aspectos relevantes relacionados con su comportamiento como la existencia del rango de funcionamiento de los motores lo cual es fundamental para comprender las limitaciones físicas del sistema y mejorar su control.

Es por ello que como mejora a la metodología implementada se propone realizar una verificación previa de compatibilidad de hardware especialmente con módulos de comunicación, así como contar con varios módulos wifi para evitar interrupciones en el proceso de trabajo del reto. Finalmente, este reto permitió comprender de manera integral la interacción de hardware y software en el Puzzlebot, así como los procesos necesarios para su configuración y comunicación, sentando las bases para los próximos retos más avanzados.

Referencias

1. University of Manchester Innovation Factory. (s.f.). Globalising Robotics Education: Introducing Puzzlebot, by Manchester Robotics.
<https://www.uominnovationfactory.com/projects/robotics-for-everyone-puzzlebot/>
2. Manchester Robotics. (2026). Puzzlebot.
https://github.com/ManchesterRoboticsLtd/TE3002B_Intelligent_Robotics_Implementation_2026/blob/main/Week1/Presentations/PDF/MCR2_Puzzlebot_Introduction_V2.pdf